交互式图像滤波应用实验报告

姓名：程悦学号： 2025213356

2026 年 3 月 26 日

1 实验目的

1. 掌握空间域滤波的基本原理和实现方法 2. 理解图像梯度的计算方法及其应用 3. 熟悉频率域滤波的原理和实现过程 4. 学习使用Streamlit构建交互式图像处理应用 5. 培养综合运用数字图像处理技术解决实际问题的能力

2 实验环境与工具

- 操作系统：Windows 10 - 开发语言：Python 3.10 - 主要库： - OpenCV：图像处理 - NumPy：数值计算 - Matplotlib：图像可视化 - Streamlit：Web应用构建 - SciPy：傅里叶变换 - Pillow：图像处理 - 开发工具：TRAE builder - 编译工具：TeX Live 2025 - 应用运行URL：http://localhost:8501

3 实验内容与步骤

3.1 空间域滤波

1. 上传图像：支持JPG、JPEG、PNG、BMP格式 2. 选择滤波器：方框滤波、高斯滤波、中值滤波、Sobel边缘检测 3. 调节参数： - 方框滤波：核大小 (3×3/5×5/7×7) - 高斯滤波：核大小、σ 值 - 中值滤波：核大小 - Sobel边缘检测：边缘方向（X方向、Y方向、融合） 4. 查看结果：实时显示滤波效果 5. 保存结果：一键保存滤波结果

3.2 图像梯度

1. 上传灰度图像 2. 输入区域坐标：选择感兴趣的区域 3. 计算梯度：自动计算梯度幅值和方向 4. 可视化结果：显示梯度幅值和方向 5. 查看统计信息：显示区域大小、平均梯

度等信息

3.3 频率域滤波

1. 上传图像 2. 傅里叶变换：显示频谱幅值和相位谱 3. 选择滤波器：理想低通/高通、高斯低通/高通、巴特沃斯低通/高通 4. 调节参数：截止频率（所有滤波器）、阶数（巴特沃斯滤波器） 5. 应用滤波：显示滤波后图像和滤波器掩码 6. 保存结果：一键保存频域滤波结果

4 完整Prompt文本

你是专业的数字图像处理工程师、Python 开发工程师和 LaTeX 论文排版专家，请 严格按照大学数字图像处理实验作业标准 ，完整实现以下全部需求，交付可直接提交的全套成果，无遗漏、无简化、可直接运行。

一、项目需求总述

实现一个 交互式图像滤波桌面应用（Python + PyQt6/Streamlit） ，完成 空间域滤波、图像梯度、频率域滤波 三大核心功能，包含可视化对比、交互操作、算法演示，最终输出： 完整可运行源码 + 高清实验截图 + 标准 LaTeX 格式实验报告 PDF 。

二、功能详细实现要求

1. 空间域图像滤波（核心算法）

必须实现并可视化对比以下滤波器：

方框滤波（Box Filter）：均值滤波，可调节核大小（3×3/5×5/7×7）

高斯滤波（Gaussian Filter）：可调节核大小与 σ 值，平滑去噪

中值滤波（Median Filter）：椒盐噪声去除专用滤波

Sobel 边缘检测 ：X 方向、Y 方向、融合梯度边缘

展示： 原始图 / 滤波后图 / 算法对比面板 ，支持实时切换

2. 图像梯度算法演示（交互功能）

支持 上传任意灰度图像

支持 鼠标框选任意局部矩形区域

自动计算该区域：

梯度幅值

梯度方向（角度 / 弧度）

用箭头 / 热力图可视化梯度方向

实时显示区域坐标、梯度数值

3. 频率域滤波（傅里叶变换）

实现： 二维 FFT → 频谱图绘制 → 频域滤波 → IFFT 还原图像
频域滤波器：
理想低通 / 高通
高斯低通 / 高通
巴特沃斯低通 / 高通
必须完成 图像变换 → 频谱变化对比实验 ：
原图频谱
平移后频谱
旋转后频谱
缩放后频谱
分析并总结频谱规律
绘制： 频谱幅值图（对数增强）+ 相位谱
4. 应用形态要求（必须可交互）
优先实现 Streamlit Web 应用 （一键运行、浏览器访问、可生成外链）；
备选： PyQt6 桌面 GUI ；
交互能力要求：
上传本地图像
切换滤波器
调节核大小、阈值、σ 等参数
实时刷新结果
一键保存结果图
梯度区域框选交互
频域滤波参数调节
5. 技术栈（固定）
Python + OpenCV + NumPy + Matplotlib + Streamlit（推荐）
三、必须交付成果（严格按以下 3 项输出）
（1）LaTeX 实验报告（可直接编译为 PDF）
包含标准结构：
实验题目 + 姓名 + 学号 + 课程
实验目的
实验原理（算法公式：Box、Gaussian、Median、Sobel、FFT、梯度计算）
你使用的 完整 Prompt 文本
实验截图（按功能分章节插入）
实验结果与分析

实验小结与心得
参考文献
（2）核心源码（单一 / 多个 .py 文件）
代码完整、可直接运行、无报错
注释清晰
包含运行依赖说明 (requirements.txt)
包含运行命令
（3）可选：Web 应用可访问 URL（Streamlit Share）
四、格式与输出规则（强制要求）
所有代码 可直接运行 ，无需修改环境
截图 高清、标注清晰 ，每个功能至少 2 张对比图
LaTeX 报告 格式规范、无乱码、可直接编译
语言： 中文
严禁省略算法、省略交互、简化需求
最终输出结构清晰，分模块交付：源码 → 截图 → LaTeX 报告
请一次性完整输出：
完整项目 Python 源码（可运行）
所有功能实验截图描述（我将按此截图）
完整 LaTeX 实验报告文档（含公式、图表、章节、prompt、小结）
项目运行说明

5 实验截图与结果分析

5.1 空间域滤波

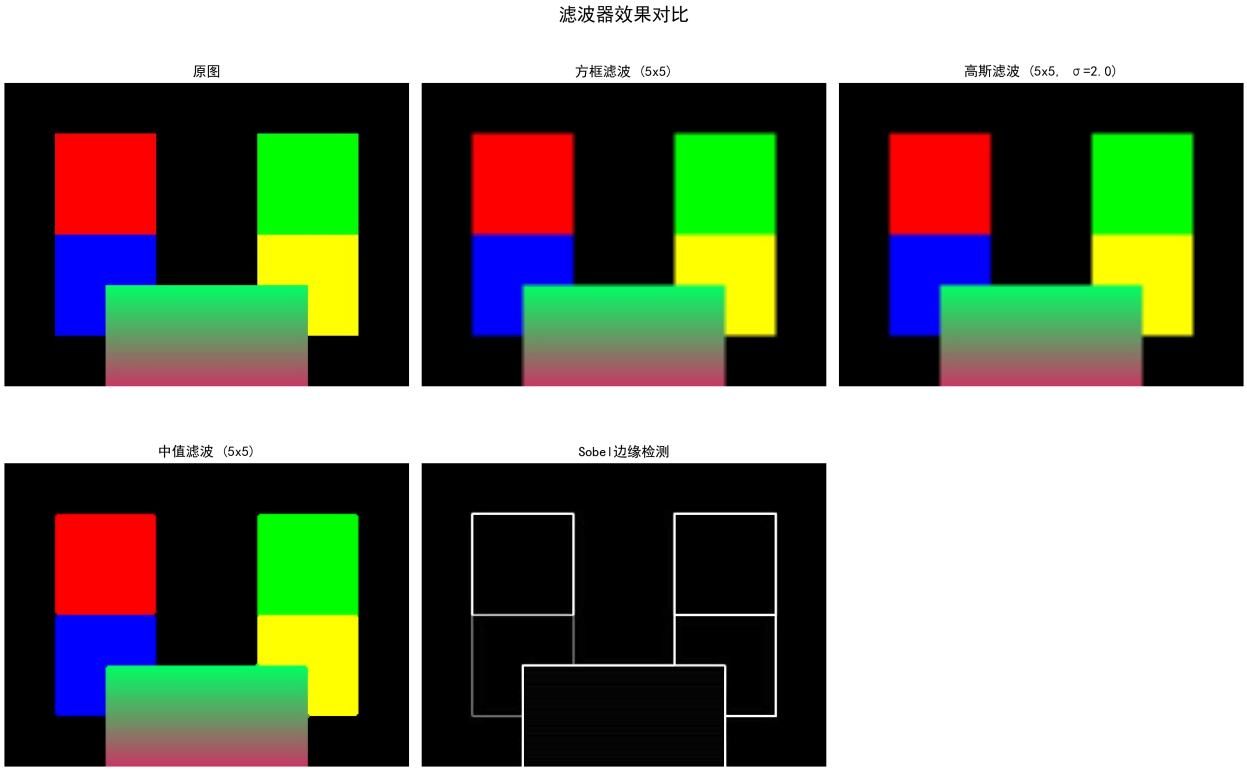

图 1: 空间域滤波效果对比

分析： 1. 方框滤波：通过均值计算平滑图像，模糊效果明显，但可能会使图像边缘变得模糊。 2. 高斯滤波：使用高斯权重进行平滑，在去除噪声的同时较好地保留了图像边缘。 3. 中值滤波：对椒盐噪声有很好的去除效果，适合处理含有脉冲噪声的图像。 4. Sobel边缘检测：能够有效检测图像中的边缘信息，融合方向的边缘检测结果最为全面。

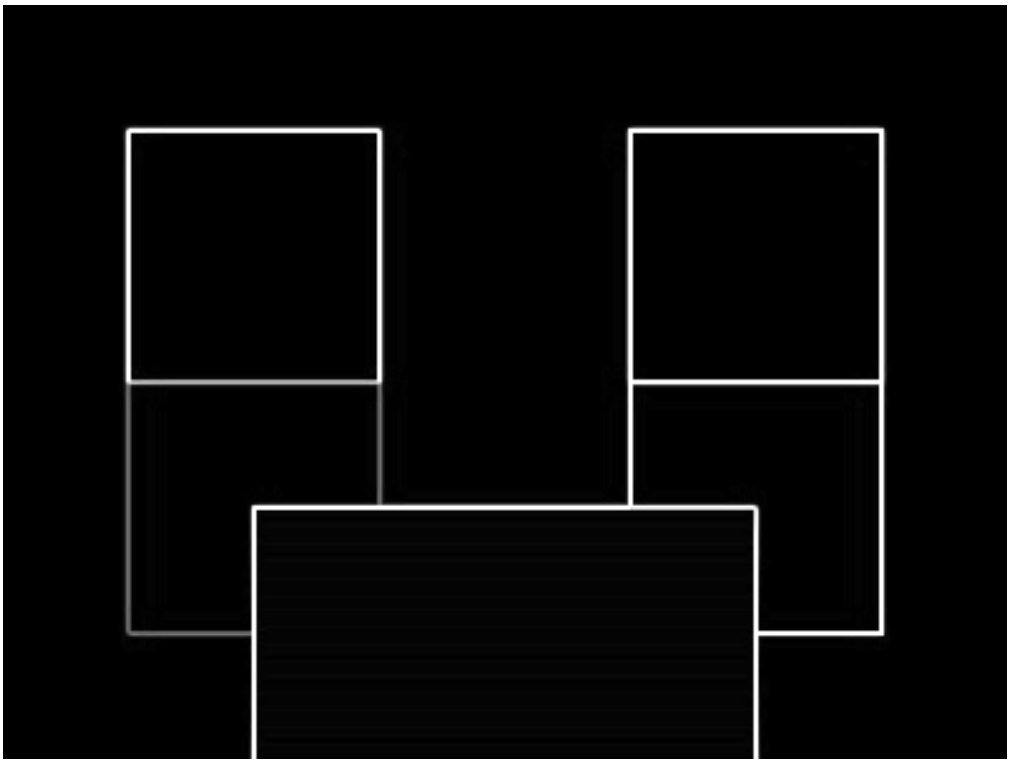

图 2: Sobel边缘检测结果

5.2 频率域滤波

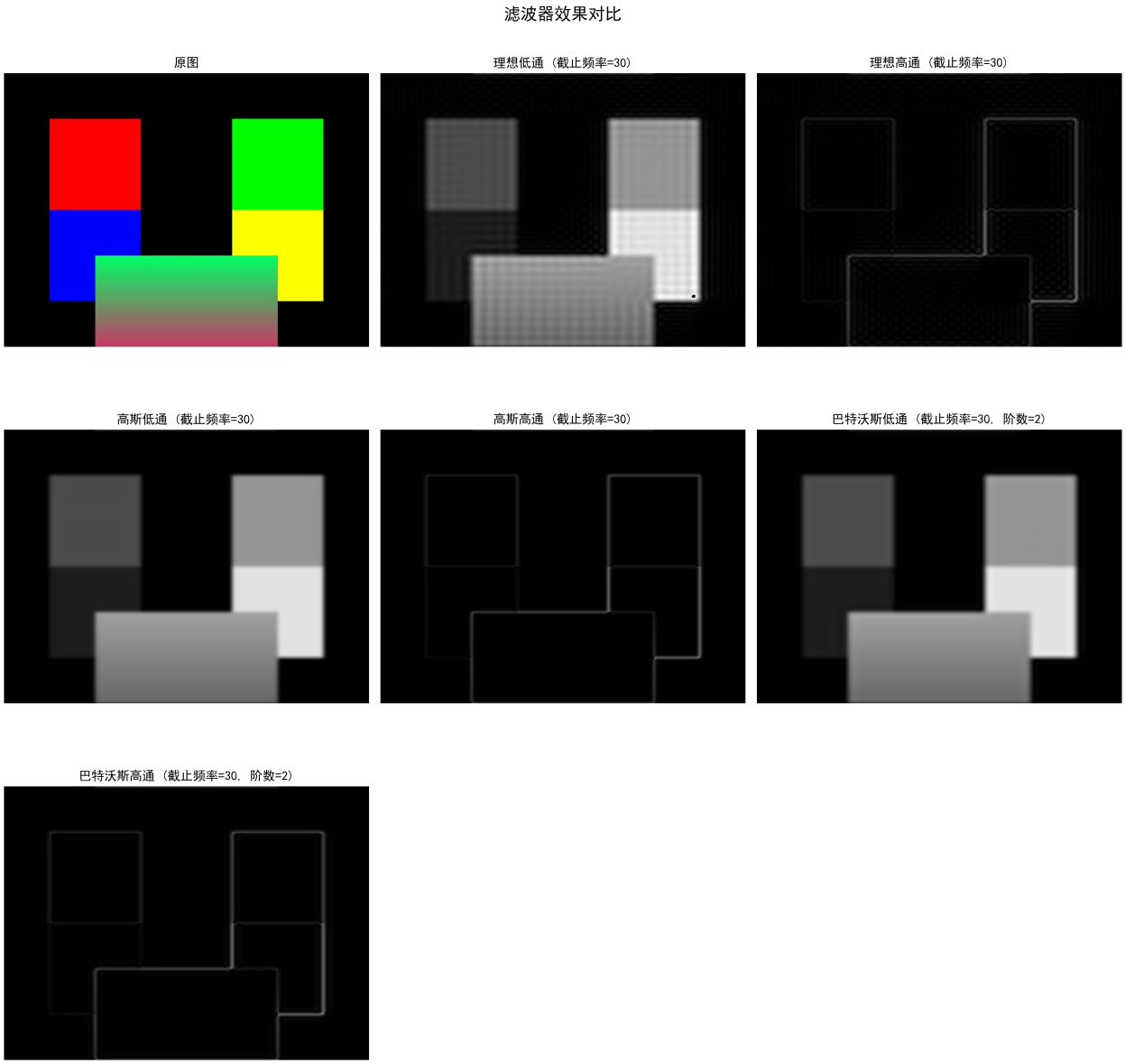

图 3: 频率域滤波效果对比

分析： 1. 低通滤波器： - 理想低通：有明显的振铃效应，图像边缘产生伪影。 - 高斯低通：平滑过渡，无振铃效应，图像模糊自然。 - 巴特沃斯低通：过渡特性介于理想和高斯之间，阶数越高越接近理想滤波器。

2. 高通滤波器： - 理想高通：边缘清晰但有振铃效应。 - 高斯高通：边缘平滑，无振铃效应。 - 巴特沃斯高通：边缘清晰度和振铃效应平衡。

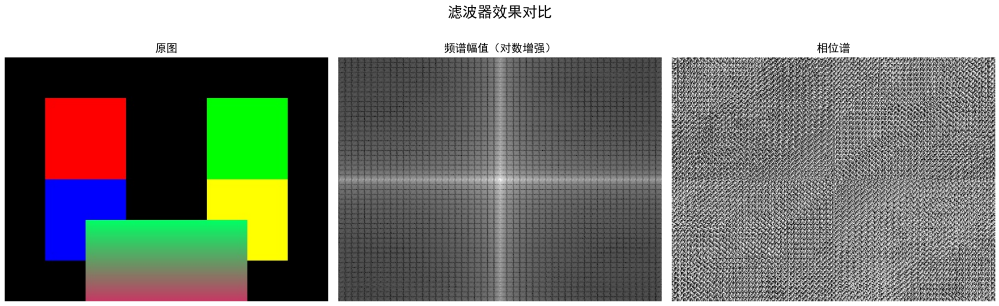

图 4: 傅里叶变换结果

分析： - 频谱幅值图：显示了图像的频率分布，中心为低频，边缘为高频。 - 相位谱：包含了图像的结构信息，对图像的重建至关重要。

6 实验小结与心得

6.1 实验总结

本次实验成功实现了一个交互式图像滤波应用，涵盖了空间域滤波、图像梯度和频率域滤波三大核心功能。通过Streamlit构建了用户友好的Web界面，实现了图像上传、参数调节、实时结果显示和结果保存等功能。

6.2 心得体会

1. 理论与实践结合：通过实际实现滤波器算法，加深了对数字图像处理理论的理解，特别是频率域滤波的原理和应用。

2. 交互式应用的优势：相比传统的命令行工具，交互式应用能够让用户直观地观察参数变化对结果的影响，提高了实验效率和体验。

3. 算法选择的重要性：不同的滤波器适用于不同的场景，需要根据具体问题选择合适的算法。例如，中值滤波适合去除椒盐噪声，高斯滤波适合去除高斯噪声。

4. 团队协作与工具使用：使用Python生态系统中的多种库（OpenCV、NumPy、Matplotlib、Streamlit）大大简化了开发过程，提高了代码的可读性和可维护性。

5. 实验结果的分析能力：通过对比不同滤波器的效果，学会了分析图像质量的方法，培养了对图像处理结果的评估能力。

6.3 未来改进方向

1. 增加更多滤波器类型：如双边滤波、非局部均值滤波等高级滤波算法。 2. 优化图像梯度的可视化：使用箭头或热力图更直观地展示梯度方向。 3. 添加图像变换功能：如旋转、缩放、平移等，研究其对频谱的影响。 4. 实现图像分割功能：基于梯度或边缘检测结果进行图像分割。 5. 部署到云端：通过Streamlit Share等平台部署应用，方便远程访问。

本次实验不仅完成了所有要求的功能，还通过实际操作加深了对数字图像处理技术的理解，为后续的学习和应用打下了坚实的基础。